

Sistema de Detecção de Fraude em Tempo Real

Projeto Teórico — Pós-Graduação

Poliana Mendes

7 de março de 2026

Resumo

Este documento apresenta o projeto de um sistema para detectar fraudes em transações de cartão de crédito em tempo real. O sistema possui duas formas de identificar fraudes: a primeira é uma verificação imediata (em até 1 segundo) que consulta uma lista de usuários, cartões e estabelecimentos já conhecidos como fraudulentos; a segunda é uma análise mais aprofundada (em até 10 minutos) feita por um modelo de Machine Learning que identifica comportamentos suspeitos. Quando o modelo identifica uma fraude nova, ele alimenta automaticamente a lista de bloqueio, prevenindo transações futuras. A arquitetura utiliza **Apache Kafka** para processar até **10.000 transações por segundo**, com disponibilidade de **99,9%** e armazenamento de dados por **180 dias**.

Sumário

1	Introdução	3
2	Modelo de Dados	3
2.1	Dados da Transação	3
2.2	Dados de Referência	3
3	Arquitetura do Sistema	4
3.1	Visão Geral	4
3.2	Por que usar o Apache Kafka?	4
3.3	Diagrama de Arquitetura	4
3.4	Componentes e Responsabilidades	5
4	Casos de Uso	5
4.1	UC-01: Transação Legítima	5
4.2	UC-02: Transação com Usuário ou Cartão Bloqueado	5
4.3	UC-03: Transação Suspeita Detectada pelo ML	6
4.4	UC-04: Analista Confirma a Fraude	6
4.5	UC-05: Analista Cancela a Suspeita (Falso Positivo)	6
5	Tecnologias Utilizadas	7
5.1	Por que MongoDB e não um banco relacional para as transações?	7
5.2	Por que Redis e não o próprio MongoDB para a lista de bloqueio?	7

6	Machine Learning: Coleta de Dados e Treinamento	8
6.1	Como os dados históricos são coletados	8
6.2	Características (features) usadas pelo modelo	8
6.3	Fluxo de treinamento do modelo	8
7	Escalabilidade e Ausência de Pontos Únicos de Falha	9
7.1	Escalabilidade para 10.000 TPS	9
7.2	Eliminação de Pontos Únicos de Falha (SPOF)	9
7.3	Disponibilidade de 99,9%	10
8	Armazenamento de Dados	10
9	Monitoramento	10
9.1	Métricas de Negócio	10
9.2	Métricas de Infraestrutura	11
9.3	Metas de Latência	11
10	Conclusão	11

1 Introdução

Fraudes com cartão de crédito causam prejuízos bilionários anualmente. Para combatê-las, é necessário um sistema que reaja rapidamente a situações conhecidas e, ao mesmo tempo, aprenda com novos padrões de comportamento suspeito.

O sistema proposto neste documento resolve esse desafio com duas camadas complementares:

- **Verificação imediata:** toda transação é checada contra uma lista de bloqueio (usuários, cartões e estabelecimentos já confirmados como fraudulentos). Se houver correspondência, a transação é bloqueada em até 1 segundo.
- **Análise inteligente:** para transações que passaram pela verificação imediata, um modelo de Machine Learning analisa o comportamento do usuário em até 10 minutos e decide se há suspeita de fraude.

As transações chegam de dois tipos de origem: maquininhas físicas (POS) espalhadas por estabelecimentos comerciais, e pagamentos online via API.

2 Modelo de Dados

2.1 Dados da Transação

Cada transação que chega ao sistema contém os seguintes campos:

Campo	Descrição
<code>transaction_id</code>	Código único que identifica a transação
<code>timestamp</code>	Data e hora exata em que ocorreu
<code>user_id</code>	Código do usuário que realizou o pagamento
<code>card_id</code>	Código do cartão utilizado
<code>site_id</code>	Código do estabelecimento onde ocorreu
<code>value</code>	Valor cobrado (em R\$)
<code>location_id</code>	Identificação da localização geográfica
<code>country</code>	País onde a transação foi realizada

2.2 Dados de Referência

Além das transações, o sistema acessa dados complementares para enriquecer a análise:

- **Usuário:** `user_id`, nome, endereço e e-mail — armazenados em banco relacional (PostgreSQL), pois são dados estruturados e raramente alterados.
- **Estabelecimento:** `site_id`, nome, endereço e categoria (bens de consumo, viagens, restaurantes etc.) — também armazenados em banco relacional.

Esses dados de referência são consultados para enriquecer cada transação antes da análise de fraude, por exemplo para saber se o estabelecimento atua em uma categoria incomum para aquele usuário.

3 Arquitetura do Sistema

3.1 Visão Geral

A arquitetura é dividida em quatro camadas, cada uma com uma responsabilidade clara:

1. **Ingestão:** recebe as transações das maquininhas e das APIs e as coloca na fila de processamento (Kafka).
2. **Resposta Imediata:** verifica a lista de bloqueio e decide aprovar ou bloquear em até 1 segundo.
3. **Análise por Machine Learning:** analisa o comportamento do usuário de forma assíncrona e identifica padrões suspeitos em até 10 minutos.
4. **Armazenamento e Monitoramento:** guarda todas as transações por 180 dias e acompanha a saúde do sistema.

3.2 Por que usar o Apache Kafka?

O **Apache Kafka** é uma ferramenta de mensageria que funciona como uma *esteira transportadora* de dados: quem produz dados (as maquininhas e APIs) coloca as mensagens nessa esteira, e quem precisa processar vai consumindo na velocidade que consegue. Isso traz três vantagens principais para este projeto:

- **Alta vazão:** suporta facilmente 10.000 transações por segundo distribuindo a carga entre vários servidores.
- **Desacoplamento:** a verificação imediata e a análise de ML consomem o mesmo dado de forma independente, sem interferir uma na outra.
- **Histórico reaproveitável:** as mensagens ficam salvas no Kafka por até 180 dias, o que permite reler transações antigas para treinar o modelo de ML.

3.3 Diagrama de Arquitetura

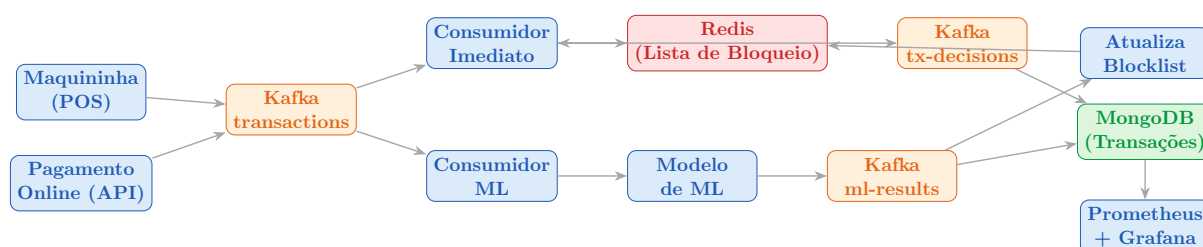


Figura 1: Diagrama de arquitetura do Sistema de Detecção de Fraude

3.4 Componentes e Responsabilidades

- **Kafka (tópico transactions):** recebe todas as transações e as distribui para os dois consumidores simultaneamente.
- **Consumidor Imediato:** lê cada transação, consulta o Redis e publica a decisão (aprovado ou bloqueado) no tópico `tx-decisions`.
- **Redis (Lista de Bloqueio):** banco de dados em memória que guarda os `user_id`, `card_id` e `site_id` bloqueados. Responde em menos de 1 milissegundo.
- **Consumidor ML:** lê a transação, calcula características do comportamento do usuário e envia ao modelo de ML.
- **Modelo de ML:** recebe as características e retorna uma pontuação de risco. Se a pontuação for alta, a transação é marcada como suspeita.
- **Atualizador de Blocklist:** quando o ML identifica uma fraude, atualiza automaticamente a lista de bloqueio no Redis.
- **MongoDB:** armazena todas as transações (normais e suspeitas) por 180 dias.
- **Prometheus + Grafana:** coletam e exibem métricas de saúde do sistema em tempo real.

4 Casos de Uso

4.1 UC-01: Transação Legítima

Situação: usuário realiza uma compra comum, sem histórico de fraude.

Fluxo:

1. A maquininha envia a transação ao sistema.
2. O Kafka repassa ao Consumidor Imediato.
3. O Redis é consultado: nenhum bloqueio encontrado.
4. Transação **aprovada** em menos de 1 segundo.
5. Em paralelo, o modelo de ML analisa o comportamento e confirma: risco baixo.
6. Transação salva no MongoDB com status **APROVADA**.

4.2 UC-02: Transação com Usuário ou Cartão Bloqueado

Situação: o `user_id` ou `card_id` já está na lista de bloqueio por fraude anterior.

Fluxo:

1. A transação chega ao Kafka.
2. O Consumidor Imediato consulta o Redis e encontra o bloqueio.
3. Transação **bloqueada** imediatamente (P50: 1 segundo).

4. O usuário é notificado (e-mail ou SMS).
5. Registro salvo no MongoDB com status `BLOQUEADA_LISTA`.

4.3 UC-03: Transação Suspeita Detectada pelo ML

Situação: o usuário não está bloqueado, mas o comportamento é anormal (ex: gastou 10x mais do que o habitual, ou realizou uma compra em outro país 30 minutos após uma compra no Brasil).

Fluxo:

1. Transação passa pela verificação imediata sem problemas.
2. O modelo de ML analisa as características e atribui pontuação de risco alta.
3. O `user_id` e `card_id` são adicionados à lista de bloqueio no Redis.
4. Transação salva com status `SUSPEITA_ML`.
5. Um ticket é aberto para revisão por um analista humano.

4.4 UC-04: Analista Confirma a Fraude

Situação: analista revisa a transação suspeita e confirma que é fraude.

Fluxo:

1. Analista confirma a fraude no painel de revisão.
2. O bloqueio no Redis é mantido.
3. O evento de confirmação é salvo como **dado de treino positivo** para o modelo de ML.

4.5 UC-05: Analista Cancela a Suspeita (Falso Positivo)

Situação: o modelo marcou como suspeita, mas o analista verifica que a transação é legítima.

Fluxo:

1. Analista cancela a suspeita no painel.
2. O bloqueio é removido do Redis.
3. O usuário é notificado da liberação.
4. O evento é salvo como **dado de treino negativo**, para que o modelo aprenda a não repetir o erro.

5 Tecnologias Utilizadas

A seguir, as tecnologias escolhidas e a justificativa de cada decisão:

Tecnologia	Por que foi escolhida
Apache Kafka	Processa grandes volumes de mensagens em tempo real com confiabilidade. Permite que vários serviços consumam o mesmo dado de forma independente e guarda o histórico de transações para retreinamento do modelo.
Redis	Banco de dados em memória com resposta em menos de 1ms — essencial para a verificação imediata da lista de bloqueio sem atrasar a aprovação da transação.
MongoDB	Banco NoSQL orientado a documentos, escolhido pela flexibilidade para armazenar transações com campos variáveis e pelo suporte a expiração automática de dados (TTL), garantindo a retenção de 180 dias sem intervenção manual.
PostgreSQL	Banco relacional para os dados cadastrais de usuários e estabelecimentos. A estrutura tabular e os relacionamentos bem definidos são ideais para esse tipo de dado estável.
Prometheus + Grafana	Ferramentas de monitoramento amplamente usadas na indústria. O Prometheus coleta métricas e o Grafana exibe dashboards visuais em tempo real.
Docker + Kubernetes	Empacotam e orquestram todos os serviços em contêineres, facilitando a escalabilidade horizontal e a substituição de componentes com falha automaticamente.

5.1 Por que MongoDB e não um banco relacional para as transações?

Um banco relacional como PostgreSQL exige que o schema (estrutura das colunas) seja definido previamente. Como as transações podem ser enriquecidas com diferentes campos ao longo do tempo (ex.: features de ML adicionadas futuramente), o MongoDB oferece mais flexibilidade. Além disso, sua funcionalidade de **TTL Index** permite configurar uma data de expiração automática para cada documento — perfeito para o requisito de 180 dias de retenção.

5.2 Por que Redis e não o próprio MongoDB para a lista de bloqueio?

O MongoDB leva dezenas de milissegundos para responder a uma consulta. O Redis, por guardar tudo na memória RAM, responde em menos de 1 milissegundo. Como a

verificação imediata precisa ocorrer em até 1 segundo e pode incluir três consultas (por `user_id`, `card_id` e `site_id`), o Redis é a única opção viável para esse SLA.

6 Machine Learning: Coleta de Dados e Treinamento

6.1 Como os dados históricos são coletados

O próprio Kafka funciona como fonte de dados históricos: como as mensagens ficam armazenadas por até 180 dias, é possível *reler* todo o histórico de transações para construir o conjunto de dados de treinamento do modelo.

Além disso, os eventos de confirmação e cancelamento de suspeitas feitos pelos analistas (UC-04 e UC-05) são salvos com seus respectivos rótulos (*fraude* ou *legítima*), formando um dataset supervisionado.

6.2 Características (features) usadas pelo modelo

Para cada transação, o sistema calcula as seguintes características que serão usadas pelo modelo:

Característica	O que representa
Frequência de compras (últimos 5 min)	Número de transações do mesmo usuário em 5 minutos
Valor comparado à média histórica	O valor atual é muito maior do que o usuário costuma gastar?
Compra em país diferente	O usuário comprou em outro país menos de 2 horas após uma compra no Brasil?
Categoria nova de estabelecimento	O usuário nunca tinha comprado nesse tipo de loja antes?
Horário incomum	A compra aconteceu em um horário atípico para esse usuário?

6.3 Fluxo de treinamento do modelo

1. **Extração:** as transações são lidas do Kafka e os rótulos (fraude/legítima) são buscados do MongoDB.
2. **Preparação:** as características são calculadas; como fraudes são raras (menos de 1% das transações), técnicas de balanceamento são aplicadas para não enviesar o modelo.
3. **Treinamento:** o modelo é treinado e avaliado com métricas como precisão, recall e F1-Score.
4. **Publicação:** o novo modelo é colocado em produção gradualmente — primeiro atendendo 5% do tráfego, depois expandindo conforme os resultados.

5. **Retreinamento:** o processo se repete semanalmente, ou sempre que as métricas de qualidade caírem abaixo do esperado.

7 Escalabilidade e Ausência de Pontos Únicos de Falha

7.1 Escalabilidade para 10.000 TPS

O sistema foi projetado para crescer horizontalmente — ou seja, adicionar mais servidores ao invés de um servidor maior. Cada componente escala de forma independente:

- **Kafka:** divide as mensagens em partições distribuídas entre múltiplos servidores. Com 64 partições, a capacidade total supera facilmente 10.000 TPS.
- **Consumidores:** cada partição do Kafka pode ser processada por um consumidor diferente em paralelo. Adicionar consumidores aumenta a capacidade proporcionalmente.
- **Redis:** opera em modo cluster, distribuindo a lista de bloqueio entre vários nós.
- **MongoDB:** divide os dados entre vários servidores (sharding) usando o `user_id` como chave de distribuição.
- **Modelo de ML:** roda como um serviço independente que pode ter múltiplas instâncias adicionadas automaticamente conforme a demanda aumenta.

7.2 Eliminação de Pontos Únicos de Falha (SPOF)

Um **ponto único de falha** é qualquer componente que, se parar, derruba todo o sistema. A tabela abaixo mostra como cada componente foi protegido:

Componente	Como o SPOF foi eliminado
Kafka	Cluster com pelo menos 3 servidores e cada mensagem replicada em 3 cópias. Se um servidor cair, os outros continuam funcionando.
Redis	Configurado em modo cluster com cópias de segurança (réplicas). Em caso de falha do nó principal, uma réplica assume automaticamente.
MongoDB	Configurado com 3 membros (1 primário + 2 secundários). Se o primário cair, um secundário é eleito automaticamente em segundos.
Consumidores	Múltiplas instâncias rodando em paralelo. Se uma falhar, o Kafka redistribui as partições para as demais.
Modelo de ML	Múltiplas instâncias atrás de um balanceador de carga. Falha de uma instância não afeta o serviço.

7.3 Disponibilidade de 99,9%

Para garantir os 99,9% de uptime (o que corresponde a no máximo 8,7 horas de indisponibilidade por ano), o sistema adota:

- **Redundância geográfica:** componentes distribuídos em pelo menos 2 zonas de disponibilidade (data centers diferentes).
- **Atualizações sem downtime:** novos deploys são feitos de forma gradual, sem desligar o sistema.
- **Modo de degradação graciosa:** se o Redis ficar temporariamente indisponível, o sistema aprova as transações e as marca para análise posterior, evitando travar os pagamentos.

8 Armazenamento de Dados

Tipo de dado	Onde é guardado	Por quanto tempo
Todas as transações	MongoDB	180 dias (expiração automática)
Transações suspeitas	MongoDB (coleção separada)	180 dias + arquivo frio
Lista de bloqueio	Redis	Sem expiração (até revogação)
Dados de usuários e estabelecimentos	PostgreSQL	Indefinido
Histórico de eventos	Kafka	180 dias

A retenção de 180 dias no MongoDB é gerenciada por um **TTL Index**: um índice especial que verifica automaticamente se algum documento passou da data limite e o remove, sem necessidade de scripts manuais.

9 Monitoramento

Para saber se o sistema está saudável, monitoramos dois grupos de métricas:

9.1 Métricas de Negócio

Estas métricas indicam se o sistema está *funcionando bem* do ponto de vista de detecção de fraude:

- **Taxa de fraudes detectadas:** porcentagem de transações bloqueadas em relação ao total. Um valor muito baixo pode indicar que o sistema não está detectando fraudes reais.
- **Taxa de falsos positivos:** porcentagem de transações legítimas bloqueadas incorretamente. Alta taxa prejudica a experiência do cliente.

- **Taxa de falsos negativos:** fraudes que passaram sem ser detectadas. Deve ser minimizada continuamente.
- **Tempo médio até a detecção:** quanto tempo leva, em média, desde a ocorrência da fraude até o bloqueio.

9.2 Métricas de Infraestrutura

Estas métricas indicam se os componentes técnicos estão *funcionando corretamente*:

Componente	O que é monitorado
Kafka	Atraso dos consumidores (consumer lag) — se estiver crescendo, os consumidores não estão dando conta do volume
Redis	Tempo de resposta das consultas e uso de memória
MongoDB	Velocidade das gravações e tamanho das coleções
Modelo de ML	Tempo de resposta do serviço de scoring

9.3 Metas de Latência

Funcionalidade	P50 (medi-ana)	P90
Resposta imediata (blocklist)	1 segundo	5 segundos
Detecção por ML	10 minutos	30 minutos

O monitoramento é feito pelo **Prometheus** (coleta de métricas) e visualizado no **Grafana** (dashboards). Alertas automáticos são disparados quando alguma métrica ultrapassa os limites definidos.

10 Conclusão

Este documento apresentou o projeto de um Sistema de Detecção de Fraude em Tempo Real que cobre todos os requisitos propostos:

- **Diagrama de Arquitetura:** apresentado na Seção 3, com os módulos e suas interações.
- **Casos de Uso:** cinco cenários detalhados na Seção 4, cobrindo transações legítimas, bloqueios imediatos, detecção por ML, confirmação e cancelamento de fraude.
- **Tecnologias:** justificadas na Seção 5, com foco especial na escolha do MongoDB e do Redis.

- **Machine Learning:** estratégia de coleta histórica e pipeline de treinamento descritos na Seção 6.
- **Sem SPOF:** redundância explicada componente a componente na Seção 7.
- **Monitoramento:** métricas de negócio e infraestrutura definidas na Seção 9.

O **Apache Kafka** é a peça central da arquitetura: ele recebe todas as transações, distribui para os consumidores em paralelo, guarda o histórico para treinar o modelo e conecta todos os módulos de forma desacoplada — garantindo que a falha de um componente não comprometa os demais.